DOCUMENT 3:
DTAF AI PIPELINE- CNC CASE STUDY

This document provides a walkthrough example using the DTAF AI Pipeline. It is separated into three sections. Section 1 provides an overview of the DTAF AI Pipeline workflow; Section 2 describes the workflow for applying DTAF AI to a CNC case study; and Section 3 lists frequently asked questions.

1. Overview of the DTAF AI Pipeline Workflow

Before using the DTAF AI Pipeline, it is helpful to understand the overall workflow. The AI architecture is structured into five layers that align with the logic of the DTAF methodology. Through these layers, the AI progressively converts an initial user-defined problem statement into structured recommendations related to DT capabilities, maturity level, simulation requirements, and trustworthiness verification and validation (V&V) needs. Table 1 presents a summary of the purpose of the individual layers, and further details on each framework itself are provided in Document 6. These categories are included here as a reference for interpreting the capability listings generated by the pipeline during the DTC CPT Layer analysis (Section 2).

Table 1: Purpose of each layer within the DTAF AI Pipeline.

Layer	**Purpose**
DTC Capability	Identifies the required DT capabilities
Data Feasibility	Determines which capabilities are realistically achievable
4R Layer	Recommends DT maturity level
4S Layer	Recommends simulation maturity
DTV Layer	Recommends V&V activities

The user only interacts with DTC Capability and Layer Feasibility layers. Once these inputs are complete, the remaining analyses are performed automatically. The overall DTAF AI Pipeline flow is illustrated in Figure 1. The pink boxes represent the knowledge databases associated with the frameworks utilized within the DTAF methodology, serving as the structured knowledge sources that guide the AI reasoning process at each stage. The green boxes represent the outputs generated by the AI after analyzing user inputs. These outputs are consolidated into a single comprehensive .html file containing the recommendations, assessments, maturity evaluations, readiness gaps, and implementation guidance needed for the user to develop the DT. A detailed explanation and walkthrough of the DTAF AI Pipeline is provided in the following sections.

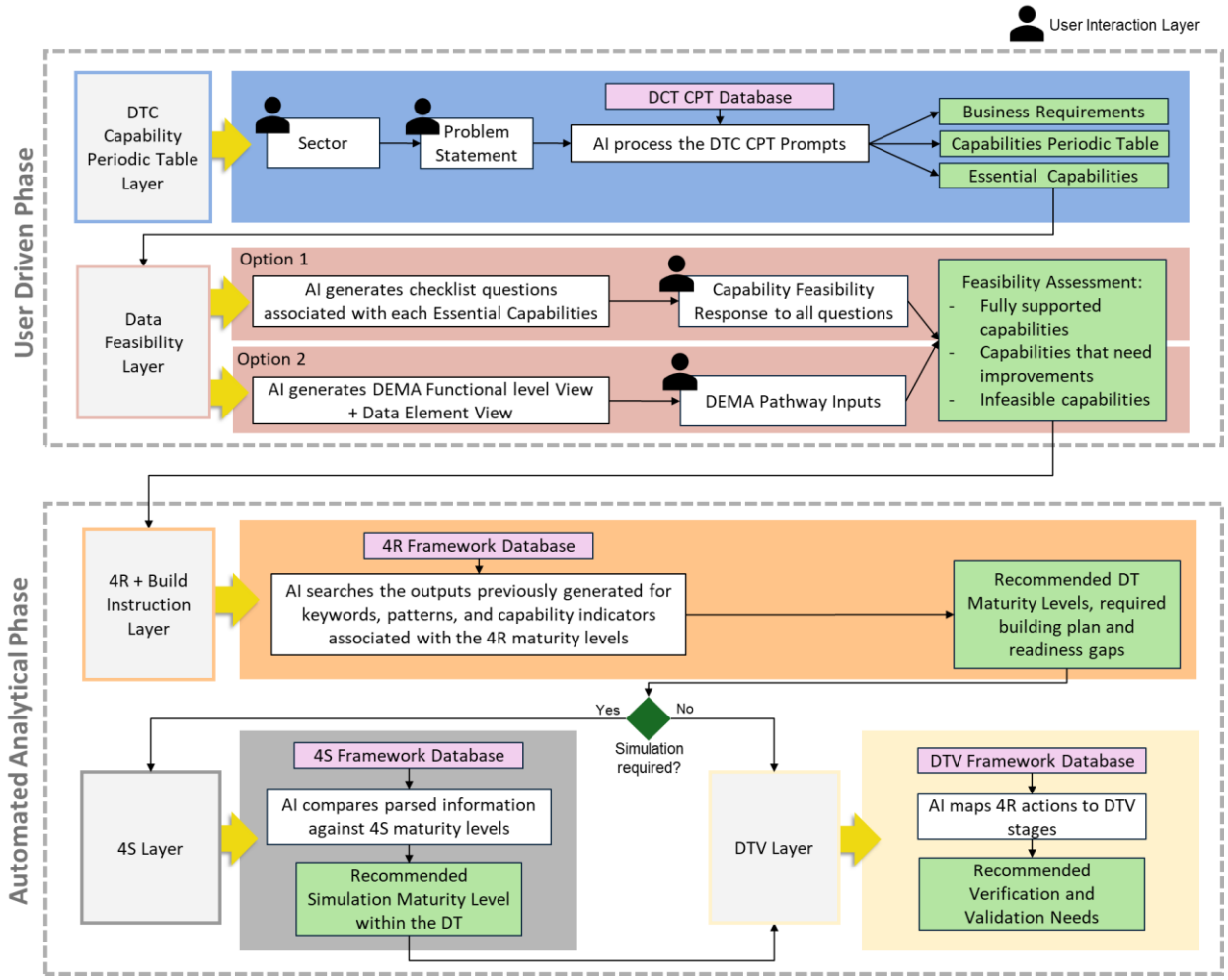


Figure 1: DTAF AI Pipeline Workflow.

2. Following Along: CNC Machining Case Study

This section demonstrates the DTAF AI Pipeline workflow using a CNC machining example. The same sequence can be followed for any manufacturing application by replacing the problem statement with one appropriate for the user's system.

STEP 1: DESCRIBE YOUR PROBLEM

Once you have followed the instructions from Document 2 and installed the DTAF AI Pipeline, you can now start interacting with it following the instructions below. During this step, the AI interprets the problem statement and identifies the DT capabilities needed to satisfy the engineering objectives.

- The AI prompts the user with the question: **"What sector should be analyzed?"**
- Type the industry sector of interest and then press **Enter**.

    - For the CNC Machining Case Study example, type "manufacturing" then press **Enter**.
- Input the **problem statement**, then press **Shift + Enter** to move to a new line. Type **END** in capital letters, and then press **Enter**.
    - The problem statement should include the main objectives that the user wants a digital twin of (e.g., reduce costs, improve efficiency, enhance safety, automate processes) and any must-have capabilities (e.g., it needs to optimize tool life, it needs to simulate, it needs a dashboard, anything and everything).
    - For the CNC Machining Case Study example, paste the following problem statement example: *We need a digital twin of our CNC machining systems to provide accurate, data-driven insight into Quality, Cost, and Time before production begins. The digital twin must determine whether we have the capability and capacity to run a given job, optimize scheduling based on available equipment, and automate key decision-making processes that currently rely on manual judgment. It should continuously monitor the degradation of cutting tools, spindles, and coolant, predict remaining useful life, and assess how component wear will impact part quality and throughput. By integrating real-time machine data with predictive simulation, the digital twin will enable proactive maintenance, reduce cycle time, extend asset life, and ensure that every job is run on the right machine, at the right time, with predictable and efficient performance.*
- *Note:* It may take 1-10 minutes for the AI pipeline to process these user inputs, depending on hardware.

STEP 2: ANSWER FEASIBILITY QUESTIONS

Once the pipeline processes the inputs from the previous step, it will identify the essential capabilities that the DT should have. It then generates a series of user-facing checklist questions associated with each capability. The total number of questions presented will depend on how many essential capabilities were identified from the earlier prompts. An example of one such checklist question is shown in Figure 2. Each letter (A, B, C, etc.) represents a specific capability element, resource, or condition listed in the prompt.

```
Capability: DS.AI (E) — Data Acquisition & Ingestion
Notes: Ensure all data sources are reliable and high-quality to support real-time decision-making.
  A) Real-time data acquisition from IoT sensors [E] [HARD]
  B) Integration with ERP and CMMS systems [E]
  C) High-quality data sources [E] [HARD]
  D) Latency less than 5 seconds for critical data [E]
  E) Scalability to handle over 50 CNC machines [E]
  F) Data quality checks in place [H]
  G) Real-time monitoring of machine health and tool usage [H]
  H) Edge computing capabilities for real-time processing [H]
Answer with letters you HAVE (e.g., A C). Or: all / none / skip
A: A
```

Figure 2: Example of a Question from the Capability Feasibility Checklist for the CNC case study.

- Select only the options that accurately reflect the current state of your system.

- Type the letter(s) corresponding to the selected options. For convenience, the system accepts the following responses:
    - A B E (example) – If only certain items apply, enter the relevant letters separated by spaces.
    - All – if all listed items are currently available
    - None – if none of the listed items are available
    - Skip – if the user is unsure or wishes to move to the next question without answering
- Press **Enter**.
- Continue answering the questionnaire following the protocol above for all the capabilities that the AI Pipeline has identified.

The AI uses these selections to conclude which capabilities are realistically supported by the physical system. In the CNC case study, the pipeline screened several Essential capabilities and the user answered "all" to each checklist question; the complete terminal transcript of these inputs is provided in the file EXAMPLE INPUTS.txt in the EXAMPLE folder of the GitHub repository.

Alternatively, the Data Element Mapping and Analysis (DEMA) pathway may be followed; see the document titled "Document 4: DEMA CNC Case Study". This pathway provides a more detailed, systematic approach to identifying, capturing, and mapping data flows across the system. Specifically, it examines the system from three complementary perspectives: the Functional-Level View, the Data Vessel-Level View, and the Data Element-Level View.

STEP 3: AUTOMATED ANALYSIS FROM THE DTAF AI PIPELINE

Given the information gathered in Steps 1 and 2, the DTAF AI Pipeline will map each capability to a phase of the 4R framework, determine the simulation capability level needed using the 4S framework, and apply verification and validation through the DTV. No user input is required during this stage. The purpose is to convert the user's responses into a structured implementation roadmap covering DT maturity, simulation requirements, and trustworthiness validation. These layers determine:

- Recommended DT maturity levels, required actions, and readiness gaps (**4R Framework**). Using a dedicated 4R framework knowledge database, the AI parses all validated information from the essential capabilities and searches for keywords, patterns, and capability indicators associated with the four maturity levels: Representation (R1), Replication (R2), Reality (R3), and Relational (R4). After the comparison, the AI identifies the highest level of maturity reasonably supported by the available evidence and generates a recommended build path, including readiness gaps and actions needed to progress further.
- Recommended simulation maturity level within the DT (**4S Framework**). Once the target 4R level is identified, the AI evaluates what type of simulation capability is needed to support that DT. Using the 4S database provided within the DTAF AI, the system compares the parsed information against the four simulation levels: Modeling (S1), Analyzing (S2), Predicting (S3), and Prescribing (S4). The AI classifies the minimum

simulation level required to effectively support the recommended 4R maturity level. It then generates guidance for achieving the selected simulation stage.

- Recommended verification and validation needs (**DTV Framework**). Using the DTV database and the outputs from the previous layers, the AI maps the generated action items to DTV stages and then produces verification and validation checks, required evidence, and measurable acceptance criteria.

STEP 4: FINAL OUTPUTS

The pipeline concludes by automatically generating the final output package, including the primary DT Build Guide (digital_twin_build_guide.html) and its supporting analysis files. The DT Build Guide consolidates all recommendations and action items produced throughout the AI pipeline into a single, user-friendly report. Specifically, it contains the actions required to build a DT to the recommended maturity level, achieve the necessary simulation capability level, and establish an appropriate verification and validation strategy to ensure the resulting DT can be trusted for decision-making.

For this CNC case study, the automated analysis recommended a DT maturity target of R2 (Replication) with a current evidence baseline of R1 (Representation), classified the required simulation level as S2 (Analyzing), and generated DTV verification and validation guidance tied to the R2 target. A complete saved run of this example is available in the EXAMPLE folder of the GitHub repository (https://github.com/lkw0012-sudo/AI-DTAF/tree/main/EXAMPLE). The file EXAMPLE INPUTS.txt contains the full terminal session, and the file EXAMPLE OUTPUTS contains every file the pipeline generated, including the final DT Build Guide. Figure 3 shows an example of the front page of the final output, which includes a high-level breakdown of the recommended DT capabilities, 4R and 4S levels, and overall guidance for DTV.

Note: Besides the DT Build Guide .html file, several additional supporting files are also generated and linked within the final HTML report for users who require deeper technical analysis. These supplementary documents include intermediate outputs generated by the model at each layer of the AI pipeline.

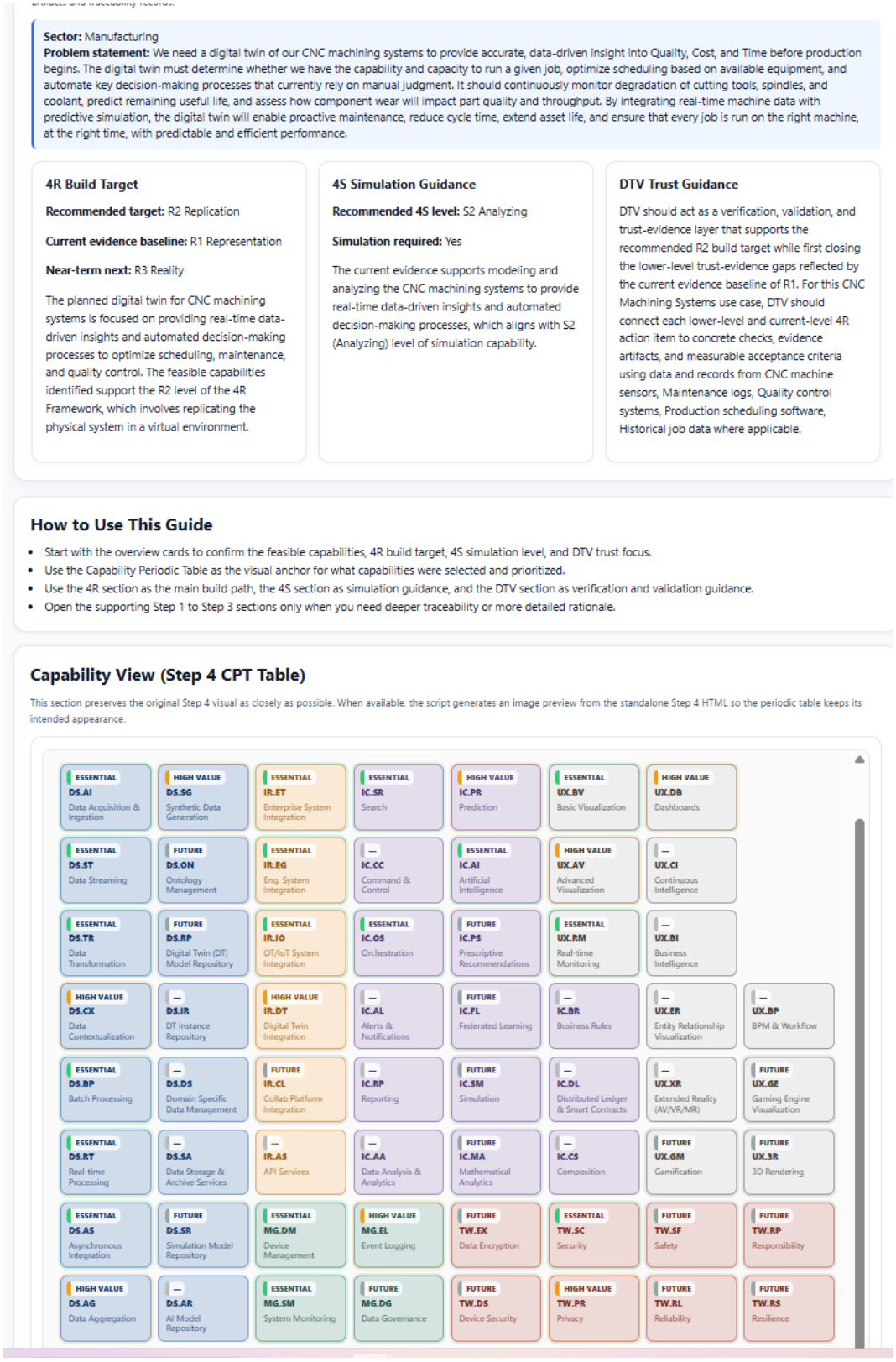


Figure 3: Example of what the front page of the primary output looks like.

3. Frequently Asked Questions

- **Why am I seeing fallback, degraded, general, or missing recommendations?**
  - This can occur when a model response is incomplete or invalid, readiness information is unavailable, or an earlier workflow stage fails or is skipped. DTAF attempts to repair invalid responses before using a clearly labeled deterministic fallback or skipping dependent stages. Because 4S and DTV depend on validated 4R results, they may be degraded or skipped when 4R is unavailable.
    - Review the Pipeline Status section of the final report.
    - Check the displayed warnings and pipeline_status.json.
    - Run the complete workflow for the most detailed results.
    - If 4R is intentionally skipped, also skip the dependent 4S and DTV stages.
    - Treat fallback recommendations as planning guidance that still requires review.
- **Why am I receiving a "File Not Found" or knowledge-base error?**
  - DTAF could not locate or read a required prompt, configuration, capability, or framework file. The program must be distributed and run with the complete project folder and not with DTAF.py alone.
    - Confirm that the complete DTAF package was extracted.
    - Keep DTAF.py, dtaf_core, the prompt files, cpt.md, model_profiles.json, and the 4R, 4S, and DTV knowledge bases together.
    - Confirm that required files have not been renamed, moved, or deleted.
    - Validate any manually edited JSON files before running DTAF again.
    - Run the program from inside the extracted DTAF project folder.
- **Why can't DTAF connect to Ollama?**
  - DTAF uses Ollama for local model generation. Model stages cannot run normally if Ollama is stopped, unreachable, or missing the selected model.
    - Confirm that Ollama is installed and running.
    - Confirm that the required model has been downloaded.
    - Verify that the model name in the command matches the installed Ollama tag.
    - Restart Ollama if the service is unresponsive.
    - Run DTAF again after confirming the connection.
    - For the recommended model, use:
      - *ollama pull gemma3:12b-it-q4_K_M*
- **Why does DTAF appear to stop while I enter the problem statement?**
  - During interactive input, DTAF waits until it receives the completion word. Afterward, the local model may also take several minutes to load and generate its first response.
    - Type the complete problem statement.
    - Type END on a separate line and press Enter.
    - Do not use Ctrl+Enter as the completion command.
    - Allow additional time for the first model request.
    - If an error or timeout appears, confirm that Ollama is running and the computer meets the model's hardware requirements.

- **Why are sections missing even though the workflow finished?**
    - Some failures affect only one part of the workflow. DTAF preserves successful results and continues when it can do so safely, while marking affected stages as repaired, fallback, degraded, failed, or skipped.
        - Review the console warnings and the report's Pipeline Status section.
        - Identify the earliest failed or skipped stage.
        - Resolve that stage's file, model, validation, or readiness issue.
        - Run the complete workflow again to regenerate dependent sections.
        - Do not interpret a completed process as proof that every stage succeeded.
- **Why do I see strange characters or symbols?**
    - Characters such as Ã¢â‚¬â€ usually indicate that a UTF-8 file was opened using the wrong text encoding.
        - Open the file in an editor or browser configured for UTF-8.
        - Save manually edited prompt, Markdown, and JSON files as UTF-8.
        - Confirm that the terminal and code editor support UTF-8.
        - Regenerate the output if corrupted characters were present in an input or prompt file.
- **Why does the final report link to an older or incorrect file?**
    - This may occur when a manually specified output directory is reused and contains artifacts from an earlier run. DTAF creates a timestamped directory by default, which helps keep runs separate.
        - Use a new output directory for every assessment.
        - Avoid reusing an old directory with --outdir.
        - Archive previous runs before generating a replacement.
        - Open the final report from the same output directory as its supporting files.
- **Why isn't the CPT table preview displayed as an image?**
    - DTAF attempts to create a PNG preview using a supported Chrome, Edge, or Chromium browser. If no supported browser is available, the original CPT HTML is embedded directly in the report.
        - Install or enable Microsoft Edge, Google Chrome, or Chromium if an image preview is required.
        - Confirm that the browser is accessible to the user running DTAF.
        - The embedded HTML table remains valid and does not indicate an assessment failure.
- **Why is DTAF having trouble reading or saving project files?**
    - Cloud-synchronized folders, security software, or insufficient permissions can temporarily lock files or delay writes.
        - Wait for OneDrive or another synchronization service to finish.
        - Confirm that the project and output directories are writable.
        - Close programs that may have the output files open. If the problem continues, move the project to a local, nonsynchronized folder and run it again.